Does it have the advanced features we want?

You may be looking only for executing basic jobs, but what happens as your application grows? A great job system will provide features like uniqueness control, rate-limiting, retry management, scheduled jobs, and workflow lifecycles.

(You may need to pay for some of these features, similar to Sidekiq's licensing model.)

Is it scalable?

The best job systems give us a ton of scalability without a ton of ceremony. In Elixir, that means expertly leveraging OTP to give us full control over scalability.

Is it observable?

We need to know what's happening in the job system. It should be observed by metrics that are deployed to an external logging system. Or it should have a dashboard so we can observe the current state of the system.

Does it support transactions (ideally, it should)?

Transactions are one of the greatest benefits of using a database. Job systems that tap into this will be easier to integrate into our system and have fewer bugs. The most popular Ruby job system—Sidekiq—is Redis-based and doesn't support transactions.

With this list of what makes a good job system, let's look at our options in Elixir and pick the best path forward.

Explore Asynchronous Jobs in Elixir

You have several options for how you can approach asynchronous jobs in Elixir. This is due to Elixir's process model—we can build simple or complex job systems inside of our existing application. The number of available options can be a bit overwhelming when you first start, so we'll look at a few common options before settling on a generic one that works for many situations.

The following examples will use a made-up piece of code that we want to make asynchronous. You don't have to type this code anywhere; it's only an example. Our scenario is that we have an HTTP endpoint that receives a data payload. We want to persist the payload to our database, but we also want to enrich the payload with data from a third-party API.

Let's jump in.